

Sistemas Distribuídos

Exame

21 de junho de 2023

Duração: 2h00m

I

- 1 Em que situações é adequada a utilização de um modelo *1-thread-por-pedido* num servidor e quais as suas vantagens? Justifique.
- 2 Identifique a dificuldade principal na serialização de estruturas de dados ligadas e resuma a técnica genérica normalmente usada para a ultrapassar.
- 3 Explique como é que o algoritmo de sincronização de relógios NTP funciona mesmo com variação da latência da rede (*jitter*).
- 4 Considere um serviço de *chat* em grupo, em que múltiplos utilizadores espalhados pelo planeta conversam fazendo perguntas e dando respostas que podem ser observadas por todos. Identifique aquele que lhe parece ser o problema de sistemas distribuídos mais importante nesta aplicação e proponha, justificando, uma solução típica.

II

Considere um sistema cliente/servidor de apoio à gestão de uma carrinha de gelados. Os clientes usam uma aplicação para pedir um gelado. Os funcionários da carrinha usam também uma aplicação para obter o próximo pedido a preparar. Consideremos que os clientes apenas podem escolher um sabor por pedido.

- 1 Apresente uma classe Java (para ser usada no servidor) que implemente a interface abaixo, tendo em conta que os seus métodos serão invocados num ambiente *multi-threaded*.

```
interface CarrinhaGelados {  
    void pede(String sabor);  
    String pedidoSeguinte();  
    List<String, Integer> topSabores();  
}
```

O método `pede` deve bloquear até o gelado estar disponível para levantamento pelo cliente. O método `pedidoSeguinte` devolve o próximo gelado a preparar, bloqueando se não estiver nenhum disponível. O método `topSabores` devolve, para cada sabor, qual o número de gelados já vendidos.

Valorização: Considere a possibilidade de se atribuir uma prioridade ao pedido, modificando a assinatura do método `pede` para `void pede(String sabor, int prioridade)`. Minimize os *threads* que são acordados.

- 2 Considere o serviço ao qual clientes e funcionários se ligam por TCP, que gere os pedidos da carrinha de gelados. Ao chegar, um cliente envia o sabor do gelado que deseja, devendo o servidor responder com `PRONTO` quando estiver disponível para recolha. Um funcionário envia `PROXIMO` para o servidor quando está disponível para preparar mais um pedido, devendo o servidor responder com o sabor a preparar. Implemente só o programa servidor usando *threads*, *sockets* TCP, e a classe desenvolvida na pergunta anterior. Use um protocolo o mais simples possível, por exemplo, baseado em linhas de texto.

Valorização: Considere que o cliente poderá ainda obter, para cada sabor, qual o número de pedidos já atendidos, enviando `TOP` para o servidor.